

CSCE 633 MACHINE LEARNING · MODULE 11 · LECTURE NOTES

Features, Data, and Labels

Where the performance actually comes from.

MODULE OUTCOMES

- Apply the standard feature transformations and say why.
- Handle missing data without destroying the signal.
- Encode categorical variables, including high cardinality.
- Explain why labels are noisy and what to do about it.
- Decide between more data, better labels, and better features.

1 Where the gains actually are

DATA WORK BEATS MODEL WORK AT THIS SCALE

Moving from a linear model to well-tuned gradient boosting is typically worth a few points of whatever metric you are using. That is a real gain and it is bounded.

Fixing a broken label definition, or discovering that two populations were pooled that should not have been, can be worth twenty. So can adding one genuinely informative feature that the domain expert could have named in the first meeting.

Yet model selection receives most of the attention, because it is tractable, measurable, automatable, and intellectually interesting, while data work is tedious, manual, and hard to put in a progress report. **The incentive gradient points the wrong way.**

So look at the data, look at the errors, and look at the labels — before training the third model. Error analysis — taking a hundred misclassified examples and categorising them by hand — is the highest-yield activity in applied machine learning and the least performed. It takes an afternoon and it routinely redirects weeks of work.

2 Transformations

Transform	When to use it	Note
Standardise (z-score)	Any distance-based or gradient-based model — linear models, SVMs, k-NN, neural networks.	Not needed for trees at all (Module 06 §1), which is one less thing to get wrong.

Transform	When to use it	Note
Log or Box–Cox	Heavy right skew, or effects that are multiplicative rather than additive.	Prices, counts, durations, incomes, populations. Frequently a large and cheap win, and it makes a linear model appropriate where it was not.
Binning / discretisation	Known meaningful thresholds, or a strongly nonlinear response you want a linear model to capture.	Discards information, and trees find the thresholds themselves , so this is mostly for linear models.
Interactions and ratios	The domain says a ratio or a product is the meaningful quantity — price per square metre, errors per request, debt-to-income.	Frequently the highest-value feature work available , and it requires domain knowledge rather than technique.
Cyclic encoding	Hour of day, day of week, month, angle, compass bearing.	Encode as sin and cos of the angle. Treating hour as an integer makes 23 maximally distant from 0 , which is exactly backwards and is a mistake people make repeatedly.
Target (mean) encoding	High-cardinality categoricals (§3).	Uses the label, so it leaks badly unless computed out-of-fold. See §3.

MISSINGNESS IS USUALLY INFORMATIVE

The fact that a value is missing frequently carries real signal. A question left unanswered, a test not ordered, a field that applies only to certain cases — the absence is a fact about the record, not an inconvenience.

So add a binary 'was missing' indicator column before imputing anything. It costs one column per affected feature and it preserves exactly the information that imputation destroys. This is cheap and is routinely skipped.

Mean imputation distorts the distribution and understates the variance — it piles mass at a single point and makes the feature look more certain than it is. It is the library default and is rarely the best choice; model-based imputation or a dedicated 'missing' category is usually better.

And dropping rows with missing values selects a biased subset. The complete cases are not a random sample — they are the records that happened to be fully filled in, which correlates with everything from data source to patient severity. **Trees handle missingness natively** (Module 06 §1), which is another substantial reason they dominate messy real-world tabular work.

3 Categorical variables

Method	Good for	Problem
One-hot	Few categories, and any linear or distance-based model.	Explodes with cardinality — a postcode field becomes thousands of columns, mostly zero.
Ordinal integer	Genuinely ordered categories, and tree models (which can split anywhere, so an arbitrary order costs less).	Implies an ordering and a spacing that may not exist, which misleads any model that treats the number as a magnitude.
Target / mean encoding	High cardinality, where one-hot is impractical. Replaces each category by the mean outcome within it.	It uses y, so computing it before the split is direct label leakage — see below.
Frequency encoding	When how common a category is matters in itself.	Collides categories of equal frequency, which may be unrelated.
Hashing	Huge or unbounded cardinality, and streaming settings where new categories appear.	Collisions are silent, and the mapping is not invertible, so interpretability is gone.
Learned embeddings	Many categories with enough data to learn from, and related categories end up nearby.	Requires a neural model (CSCE 636) and enough examples per category.

Target encoding is the one that will burn you. Computing each category's mean outcome over the whole dataset and then cross-validating gives every row a feature that was partly computed from its own label — which is Module 04 §2's leakage in a form that looks like feature engineering rather than like a mistake. **The correct version computes each fold's encoding from the other folds only**, with smoothing toward the global mean for rare categories. **It is subtle enough to be worth implementing once by hand** before using a library's version.

4 Labels

LABELS ARE NOISIER THAN ANYONE ADMITS

Human annotators disagree with each other routinely — 5 to 20% of the time on tasks that are described in the instructions as objective, and far more on anything involving judgement.

Standard benchmarks contain label errors. A systematic audit of ten widely-used test sets — including ImageNet and MNIST — found an average of 3.4% of test labels wrong, **and correcting them changed the relative ranking of published models.** The benchmarks that the field's progress was measured against were themselves imprecise.

So your test set's ceiling is the label quality, and a model reported as 'exceeding human performance' may be fitting the annotators' systematic biases rather than the underlying truth — which is a different and much less impressive achievement.

Measure inter-annotator agreement before trusting any of it. If two trained humans agree only 80% of the time, then 85% accuracy is not a meaningful target and the difference between a model at 82% and one at 84% is not measurable. **This number bounds everything else and is almost never reported.**

- **Measure agreement first.** Cohen's kappa for two annotators, Fleiss' for more, computed on a sample that was deliberately double-annotated. Budget for this at the start; retrofitting it is expensive.
- **Write a labelling guide and revise it when annotators disagree. Most disagreement is definitional rather than perceptual** — the annotators are applying different rules, not seeing different things — and the fix is a clearer rule, not better annotators.
- **Audit the examples the model gets most confidently wrong. A large share of them are mislabelled rather than genuinely hard,** and this audit is nearly free — sort by loss, read the top fifty. **It reliably finds both label errors and leakage** (Module 12) and is the single cheapest diagnostic in this module.
- **Weak supervision.** Combine several noisy heuristic labelling functions and model their individual accuracies and correlations to produce probabilistic labels — which lets you label large datasets without annotating them one at a time.
- **And active learning.** Label the examples the model is most uncertain about (Module 10 §3), because that is where an additional label changes the model most — though beware that the resulting labelled set is no longer a random sample, which complicates evaluation.

WHERE TO SPEND THE NEXT WEEK

Is the learning curve still descending? Then more data, and you can estimate how much from the curve's slope (Module 03 §1).

Have the curves converged, both high? Then better features, or more capacity — and features first, because they reduce bias without adding variance.

Are the errors concentrated in one category or one segment? Then that segment needs features or data, and a different model architecture will not touch it. **This is what error analysis reveals and nothing else does.**

Are the confident errors mostly mislabelled? Then fix the labels — and note that doing so moves the measured ceiling, so previous results are no longer comparable. **And if nothing is obvious, do the error analysis properly before tuning further:** a hundred misclassified examples, read and categorised by hand, on paper. **It is the least glamorous and most reliably valuable thing in this course.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Northcutt, Athalye & Mueller — Pervasive Label Errors in Test Sets (free)	The §4 audit. Ten benchmarks, the error rates, and the ranking changes. Read it before trusting any benchmark number.
Andrew Ng — Machine Learning Yearning, and the data-centric AI material (free)	The §1 argument and the error-analysis procedure, stated as an engineering discipline.
Zheng & Casari — Feature Engineering for Machine Learning	The §2 and §3 catalogue, with the reasoning for each transform.
Ratner et al. — Snorkel: Rapid Training Data Creation with Weak Supervision (free)	The §4 weak supervision approach, with the accuracy-modelling machinery.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Do an error analysis:** take 100 misclassified examples from your project, read them, and categorise the failures.
- 2 Report what fraction were mislabelled, ambiguous, or genuinely hard.
- 3 Apply a log transform to a skewed feature and compare model performance before and after.
- 4 **Encode hour-of-day as an integer and as sin/cos,** and compare a distance-based model's performance.

- 5 **Add missingness indicators** before imputing and measure the difference.
- 6 Compare dropping incomplete rows against imputing, and **check whether the complete cases differ systematically**.
- 7 **Implement out-of-fold target encoding by hand**, then compute it naively and compare the cross-validation scores.
- 8 **Measure inter-annotator agreement** on a sample of your project's labels, double-annotating fifty examples.
- 9 **Sort your training set by loss and read the top fifty**. Report how many are label errors.
- 10 Decide, with evidence, where to spend your next week on the project.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does data work usually beat model work, and why is it under-performed?
- 2 Give six feature transforms and when each applies.
- 3 Why must cyclic quantities be encoded with sin and cos?
- 4 Why is missingness informative, and what should you do before imputing?
- 5 Why does dropping incomplete rows bias the sample?
- 6 Give six categorical encodings and the problem with each.
- 7 Why does target encoding leak, and what is the correct version?
- 8 How noisy are benchmark labels, and what does that bound?
- 9 Give five ways to work with imperfect labels.